Alternatively, using the .NET Fluent Assertions library, you could write something like this:

```
memberAges.Should().Contain(item => item < 18);
```

Traditionally, higher-level assertions like this would often require loops and nontrivial logic in the unit tests. This isn't only risky; it's also often enough to discourage developers from testing nontrivial outcomes. By making it easier for developers to express their expectations effectively, fluent assertion libraries contribute to more meaningful and higher quality executable specifications.

Hamcrest and FEST-Assert play similar roles in Java-based BDD. Hamcrest is more flexible and easier to extend, but FEST-Assert has a simpler syntax and is a little easier to use. The constraint-based assert model in NUnit is similar, and it has a particularly rich library of assertions. And the Fluent Assertion library proposes an expressive style of assertions with a very BDD feel to it. All of these are a vast improvement on the traditional assert statements.

Fluent assertion libraries are in no way specific to BDD, and they can be used to make any unit tests easier to understand. But their emphasis on readability, expressiveness, and communication makes them well aligned with the BDD philosophy.

10.6 Summary

In this chapter you learned about practicing TDD techniques using a BDD style:

- Low-level BDD can be considered as an extension of classic TDD, or even "TDD practiced very well."
- BDD acceptance criteria lead to lower-level BDD unit tests, and help you discover the detailed application design.
- BDD unit-testing tools exist for virtually all modern languages and platforms, with the RSpec family of tools being very widespread.
- More recent BDD tools like Spock and Specs2 are more powerful and expressive.
- Fluent assertion libraries make it easier to express your expectations clearly.

In the next chapter, we'll look at how BDD fits into the broader picture of project management and reporting, and how to get the most out of your living documentation.